Pyroclast: GPU-Accelerated Monte Carlo Simulation for Lava-Flow Habitat Impact Assessment

Emanuele Galiano

May 24, 2026

Contents

Chapter 1

Kernel Monte Carlo

In this chapter we present the design and implementation of the kernel Monte Carlo method used in Pyroclast for simulating lava flow and assessing habitat impact. In the first section we discuss the mathematical formulation...

1.1 Mathematical Formulation

The problem consists of estimating the probability that a lava flow will cause the destruction of a specific habitat area, given a pre-calculated map of invasion probabilities.

1.1.1 Variables and Parameters

We can define the variables of the problem as follows:

- Let N_c be the total number of active cells that make up the habitat under evaluation.
- Let $P = \{0_0, p_1, \ldots, p_{N_c-1}\}$ be the set of invasion probabilities for each cell of the habitat, where p_i is the probability that the lava flow will invade cell i.
- Let $\theta \in [0, 1]$ be the critical fraction or destruction threshold, which represents the minimum fraction of the habitat that must be invaded for it to be considered destroyed.
- Let R be the total number of Monte Carlo simulations to be performed.

1.1.2 The Monte Carlo Run

For a single simulation identified by the index r,where $0 \leq r < R$, we perform the following steps:

1. **Sampling**: For each individual cell k of the habitat, we draw a pseudo-random number $X_{r,k} \sim \mathcal{U}(0, 1)$.
2. **Invasion Evaluation**: Cell k is considered "invaded" by lava in simulation r if the draw value is less than or equal to its invasion probability, i.e., if $X_{r,k} \leq p_k$, and it can be formalized with an indicator function

$$I_{r,k} = \begin{cases} 1 & \text{if } X_{r,k} \leq p_k \\ 0 & \text{otherwise} \end{cases} \tag{1.1}$$

3. **Destruction Assessment**: We first define the total number of invaded cells in simulation r as

$$C_r = \sum_{k=0}^{(N_c-1)} I_{r,k} \tag{1.2}$$

and then the habitat is considered destroyed in run r if the fraction of invaded cells exceeds the critical threshold θ (normalized by the total number of cells):

$$D_r = \begin{cases} 1 & \text{if } \dfrac{C_r}{N_c} \geq \theta \\ 0 & \text{otherwise} \end{cases} \tag{1.3}$$

1.1.3 Monte Carlo Estimator

The ultimate goal of the parallel exeuction is to compute the total number of simulations that resulted in habitat destruction. This requires summing D_R, over all R runs. The Monte Carlo estimator for the probability of habitat destruction, also the overall probability of destruction $\hat{P}_{\text{destruction}}$, can be estimated as:

$$\hat{P}_{\text{destruction}} = \frac{1}{R} \sum_{r=0}^{R-1} D_r \tag{1.4}$$

1.2 Standard Kernel

The first version of the Monte Carlo kernel implemented in Pyroclast is a straightforward parallelization of the algorithm described in the previous section. Each simulation run is executed independently, and the results are aggregated by a reduction operation at the end of the kernel execution.

1.2.1 Kernel Design

The kernel operates on a 1-dimensional grid of work-items, where each work-item is reponsible for executing a whole Monte Carlo simulation on the full array of invasion probabilities.

Work-Item Mapping A critical architectural decision in this baseline was how to map the computational workload to the GPU's work-items. The most intuitive approach is the pure stream processing model, where each work-item is responsible for executing only single Monte Carlo. This approach was delibartely discarded due to a sever hardware inefficiency known as the "prime number problem"[1]: in OpenCL the number of launched work-items is defined by the Global Work Size, which is hwardware-partitioned into smaller blocks called work-groups, defined by the Local Work Size. If we tie the number of work-items to the number of simulations, an user requesting a number of simulations that is prime would force a primw GWS and since the only integer divisors of a prime number are 1 and itself, the hardware would be forced to launch work-groups containing a single work-item (LWS = 1).

[1] Also knowns as one of the "leaky abstractions" of GPU programming.

Sliding Window To resolve this limitation a sliding window approach is adopted. Instead of launching a grid sized perfectly to the dataset, we launch a hardware optimized grid and allow each work-item to process multiple runs through a for loop, taking strides[2] equal to the entire GWS.

1.2.2 Algorithmic Complexity

The theoretical time complexity of the kernel depends on the total number of Monte Carlo simulations R, the number of cells N_c and the number of work-items P launched.
AT THE END, for now just notes.

1.3 Benchmark

[2]The stride represents the number of simulations processed by each work-item in the sliding window approach.

Appendix A

Library Architecture

(Appendix placeholder — to be expanded.)
DUMMY CITATION [1]

Bibliography

[1] Harry Percival and Bob Gregory. *Architecture Patterns with Python: Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices.* O'Reilly Media, Sebastopol, CA, 1 edition, 2020.